Kaip sukurti sudėtingą objektą su daugeliu parametrų?

Kaip gauti dar vieną tokį patį objektą?

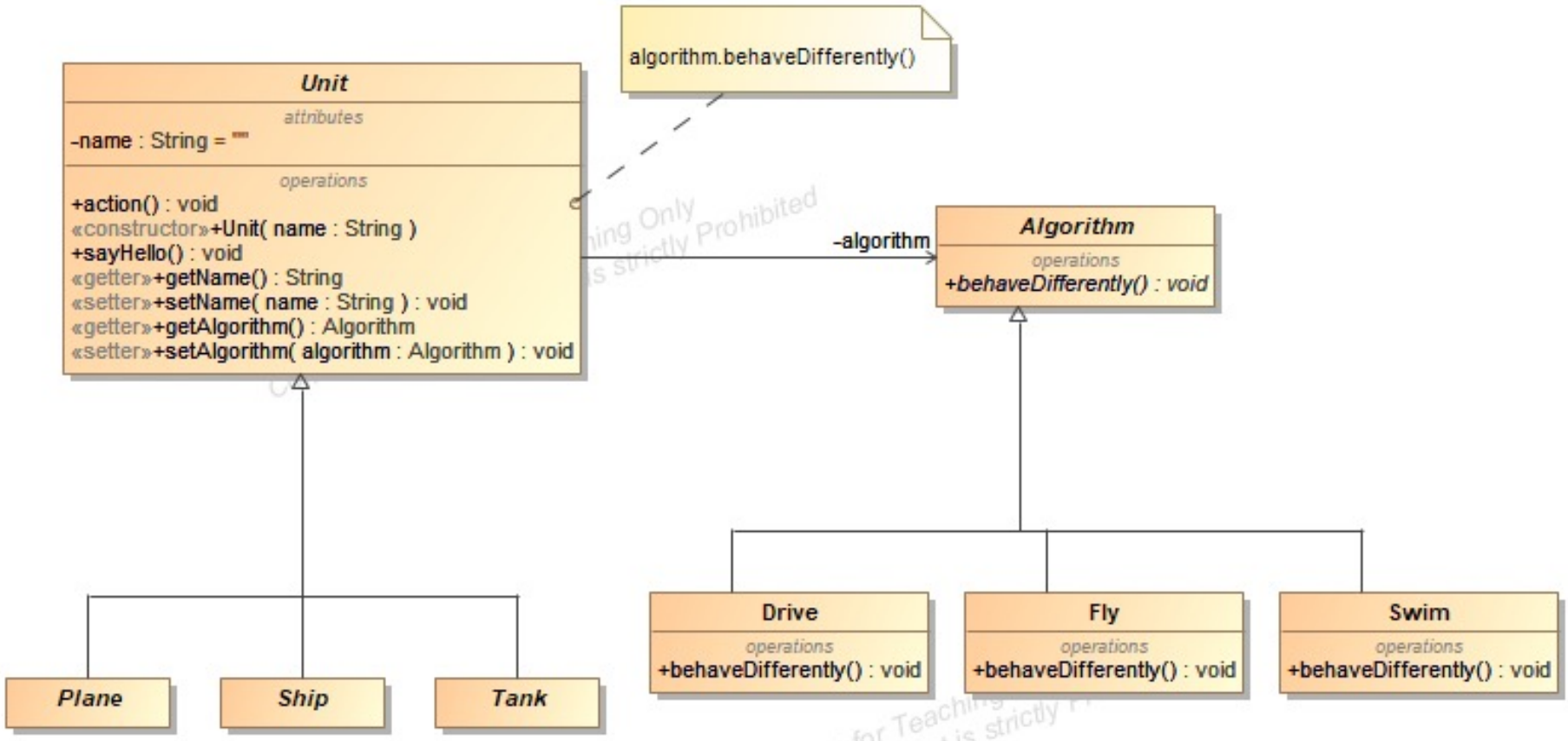
T120B516 Objektinis programų projektavimas

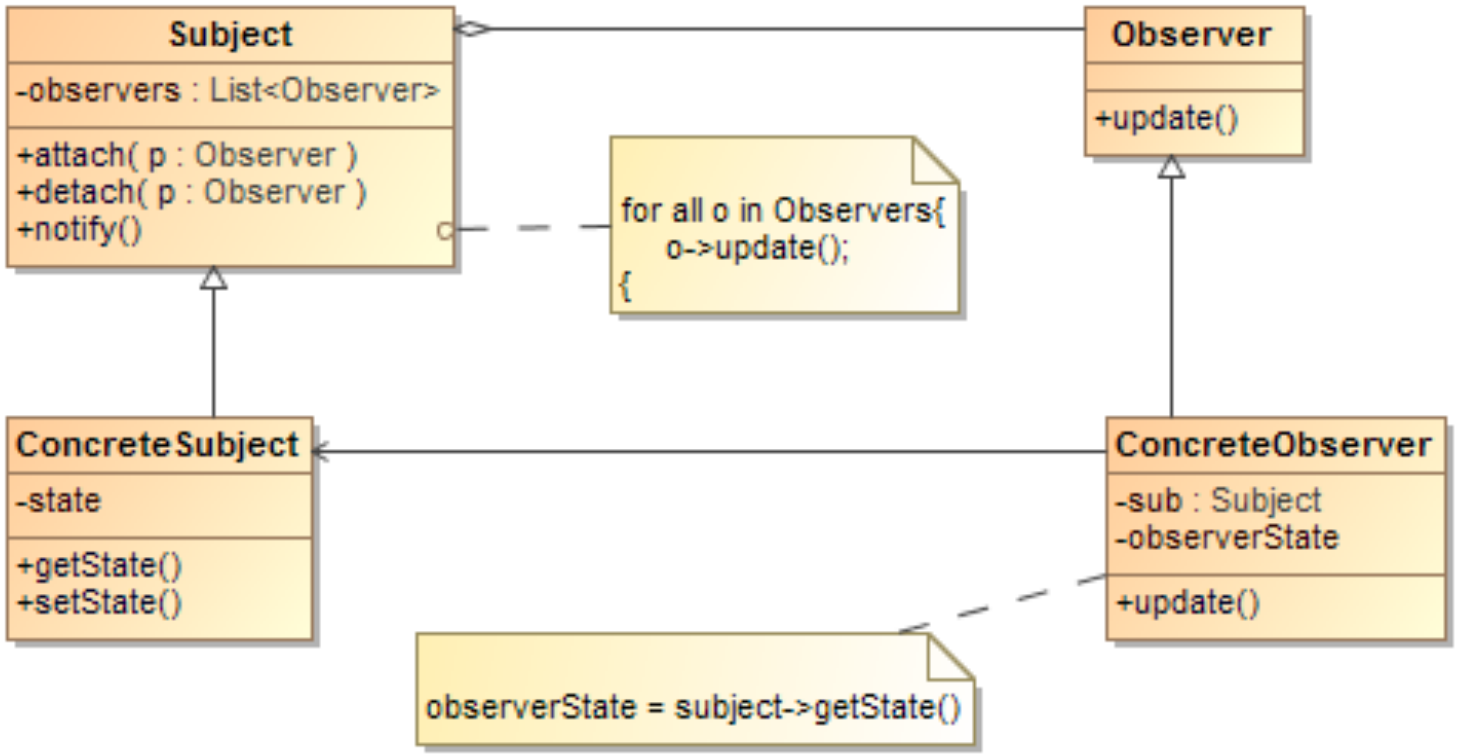
2026 rudens semestras

Dr. Andrej Ušaniov

Ką prisimenate iš praėjusio karto?

- **Strategy** - Kaip pakeisti objekto elgseną?
- **Observer** - Kaip apie pokytį informuoti kitus objektus?


algorithm.behaveDifferently()
Unit
attributes
-name : String = ""
operations
+action() : void
«constructor»+Unit(name : String)
+sayHello() : void
«getter»+getName() : String
«setter»+setName(name : String) : void
«getter»+getAlgorithm() : Algorithm
«setter»+setAlgorithm(algorithm : Algorithm) : void
-algorithm
Algorithm
operations
+behaveDifferently() : void
Plane
Ship
Tank
Drive
operations
+behaveDifferently() : void
Fly
operations
+behaveDifferently() : void
Swim
operations
+behaveDifferently() : void


Subject
-observers : List<Observer>
+attach(p : Observer)
+detach(p : Observer)
+notify()
for all o in Observers{
o->update();
{
Observer
+update()
ConcreteSubject
-state
+getState()
+setState()
ConcreteObserver
-sub : Subject
-observerState
+update()
observerState = subject->getState()

Sukurkime lėktuvą

```
new Plane(
     "blue",
     4,
     true,
     "120mm",
     200,
     false,
     null,
     ...
);
```

Ar žinote, ką reiškia septintas parametras?

O jeigu pusė parametrų neprivalomi?

- `new Plane("blue", 4, true, null, 200, false, null);`

Kaip po mėnesio suprasite, ką reiškia true, null, 200, false?

Kodėl tiesiog nepridėti dar vieno konstruktoriaus?

- `Plane(...)`
- `Plane(..., engine)`
- `Plane(..., engine, wheels)`
- `Plane(..., engine, wheels, seats)`
- `Plane(..., engine, wheels, seats, weapons)`

Telescoping Constructor Problem

Builder - Kaip sukurti sudėtingą objektą žingsnis po žingsnio?

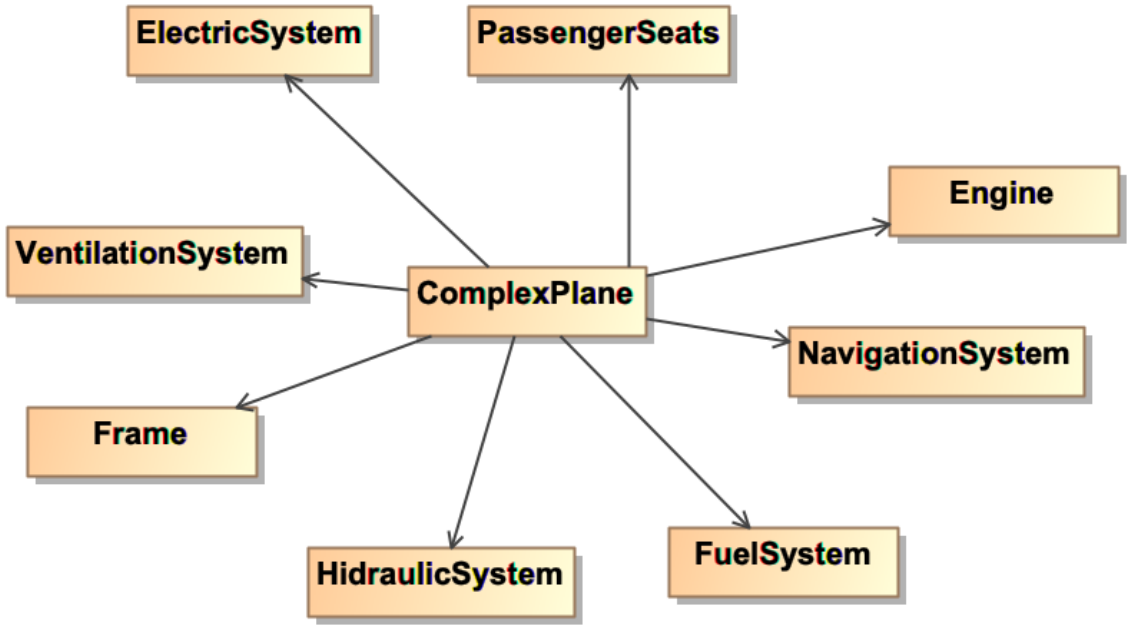
- Type: Creational

- Also Known As:

- Intent: “Separate the construction of a **complex object** from its representation so that the same construction process can create different representations.”

What is a complex object?

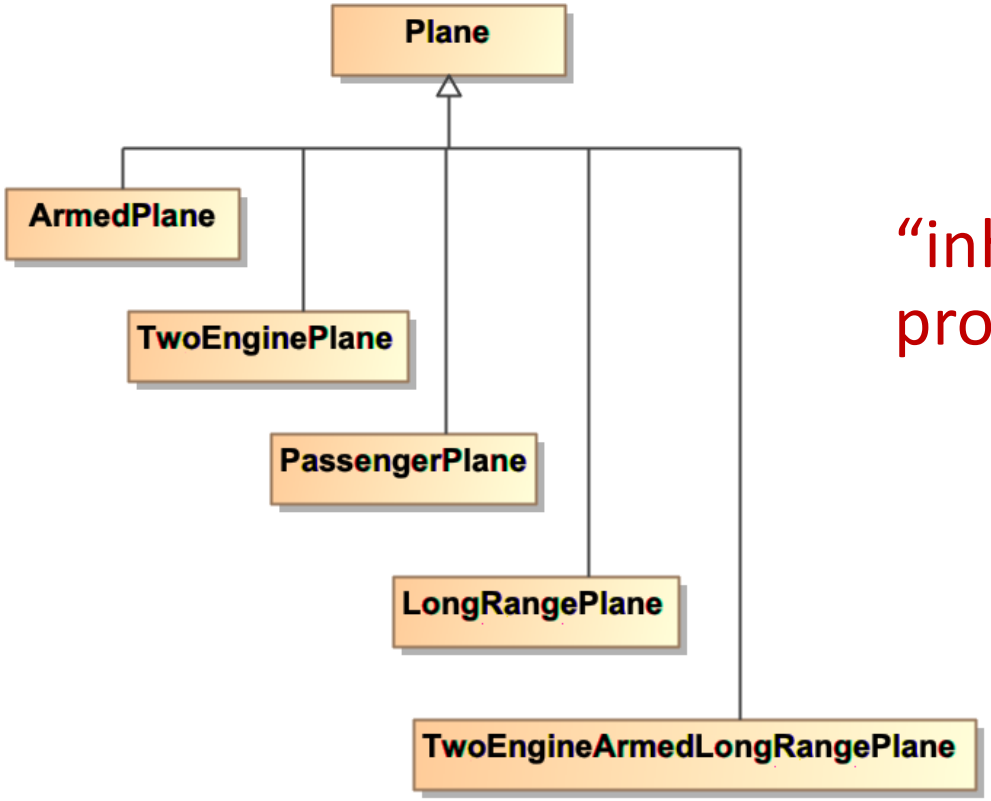
Sudėtingas objektas nebūtinai turi tik daug parametrų. Dažnai jis pats susideda iš kitų objektų.



Taigi, kada mums praverčia Builder?

- objektas turi **daug konfigūracijos parametrų**;
- dalis parametrų yra neprivalomi **optional**;
- tuo pačiu procesu norime sukurti **skirtingas produkto versijas**.
- objektą norime kurti **žingsniais**;

Builder: an example from the real world

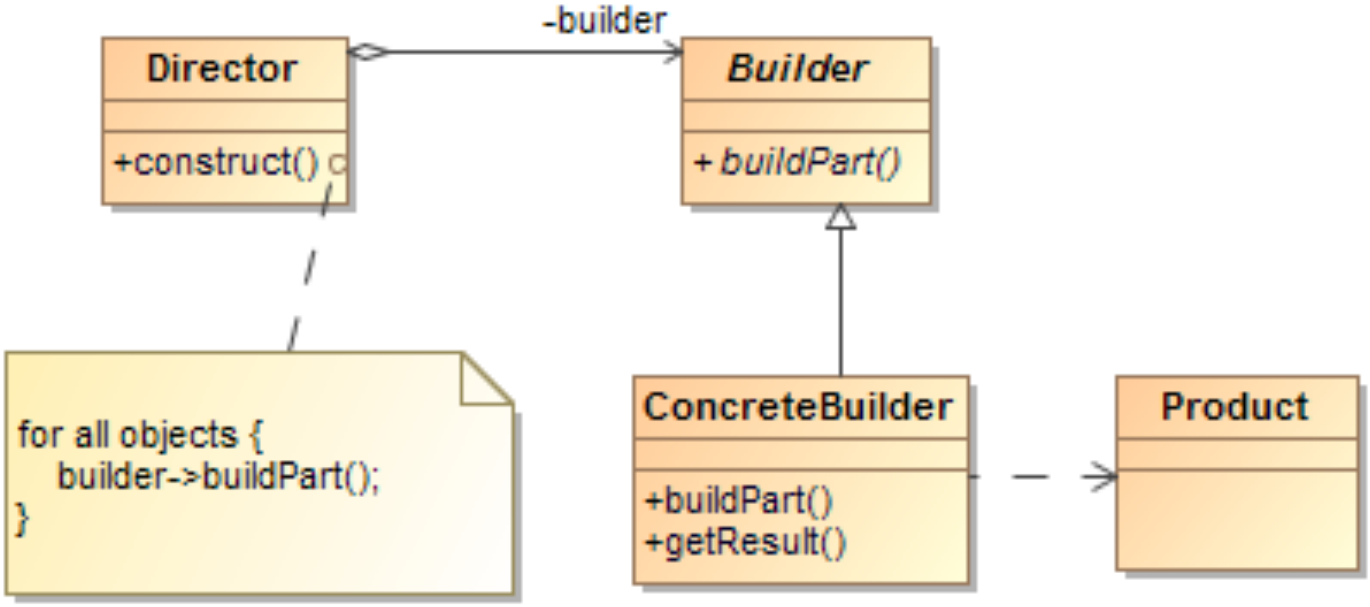


"inheritance explosion" probleme

Kaip GoF sprendžia skirting lektuvų problem?

Builder žino KAIP sukurti dalį.

Director žino KOKIA TVARKA kviesti žingsnius.



Kuris variantas aiškesnis?

```
new Plane(
      "blue",
      4,
      true,
      "120mm",
      200,
      false,
      null,
      ...
);
```

```
Plane plane = new PlaneBuilder()
.color("blue")
.wheels(4)
.cannon("120mm")
.wings(…
```

Kodėl Builder metodai grąžina patį Builder?

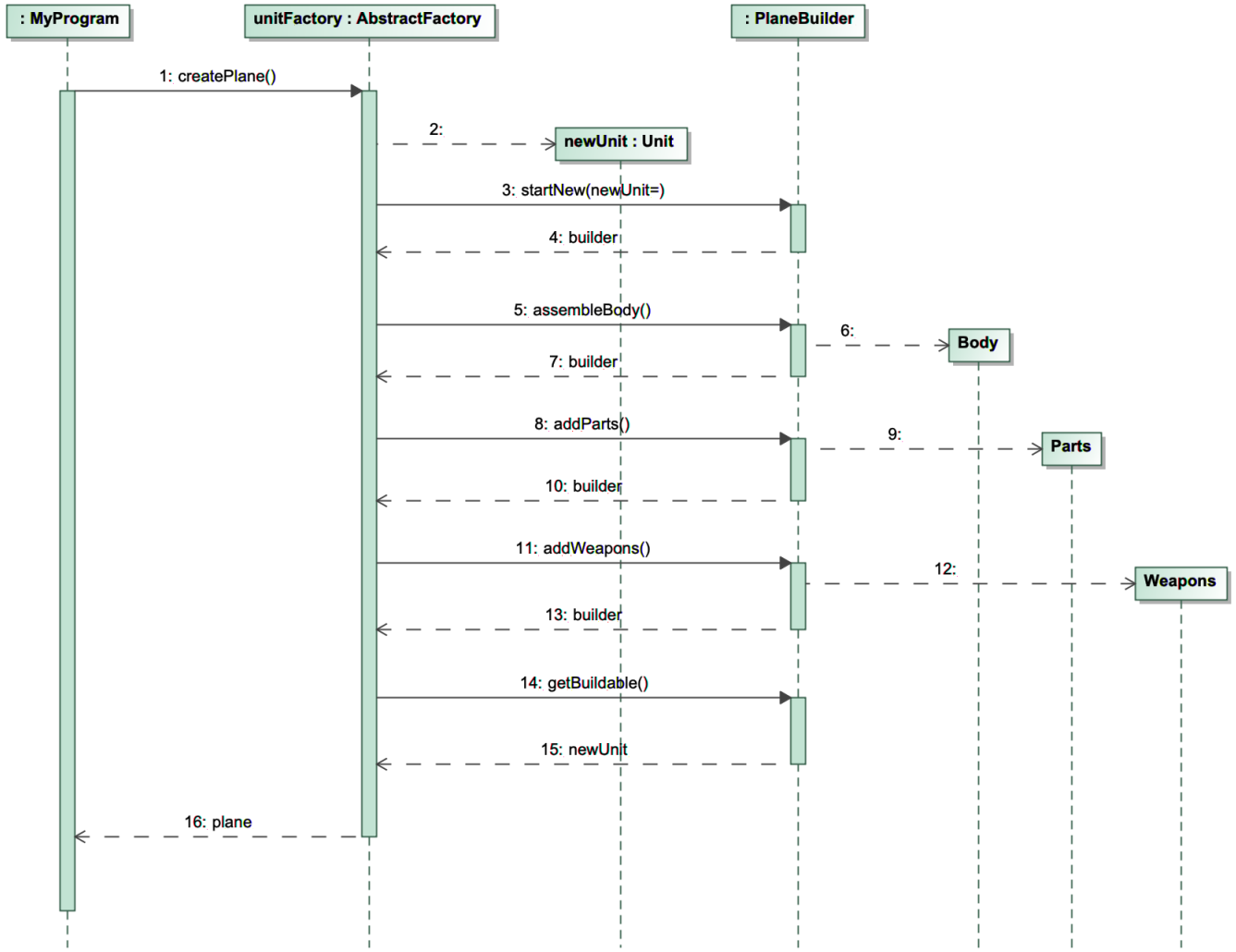
Kodėl po `.addWings()` galiu iš karto rašyti `.addWeapon()`?

Kas nutiks, jei iškviesiu .build(), nors lėktuvas dar neturi sparnų?

- Tikrinimas build() viduje ir klaidos išmetimas (exception) arba Empty result grąžinimas;
- Reikalauti privalomų parametrų per konstruktorių;
- Naudoti interfeisų grandinę (step builder), tuomet build() bus galima iškviesti tik po visų privalomų metodų.

Kas vyksta nuo pirmo .addPart() iki .build()?

- Builder sequence



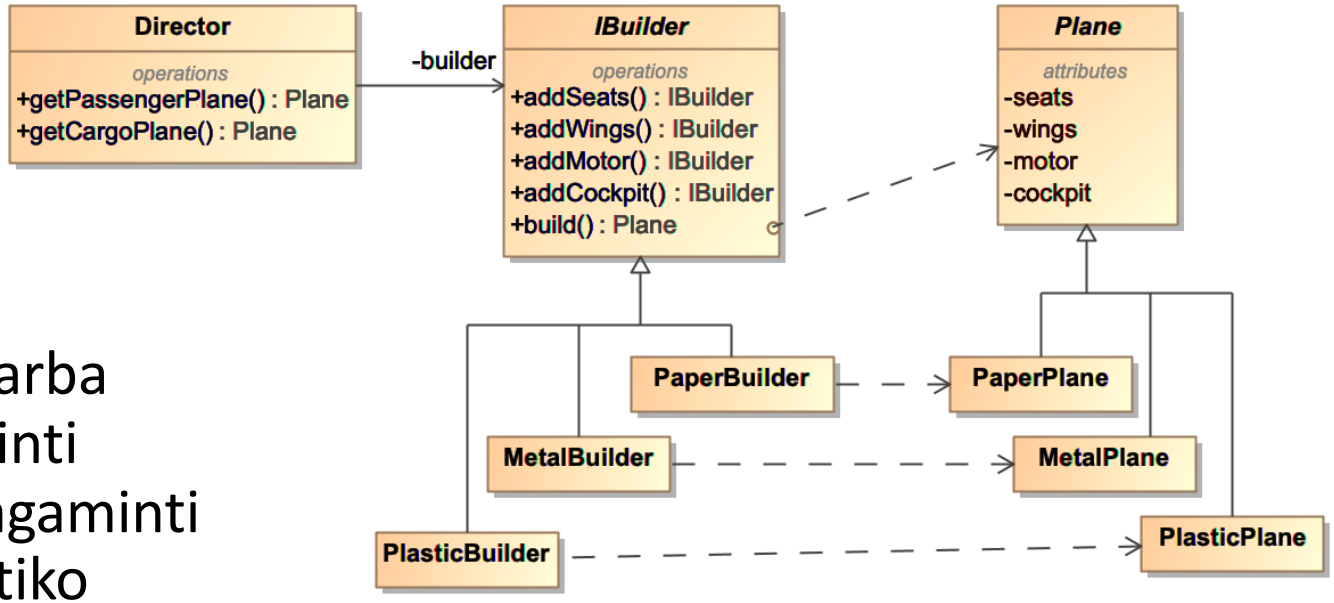
Builder baigė vieną objektą. Kaip suprasti, kad jau statome kitą?

O kada pradedamas naujas lėktuvas?

`Builder.startNew()` arba `reset()`

```
Unit unit = new Builder.startNew()
        .AssembleBody()
        .AddParts()
        .AddWeapons().getUnit();
```

Jei turime tuos pačius konstravimo žingsnius, ar visada gausime tą patį rezultatą?



- Builderis – statybininkas arba surinkėjas, žino kaip gaminti atskiras dalis. Gali pvz. Pagaminti iš metalo, medžio ar plastiko
- Direktorius – vadovauja procesui

Kada objektas jau „pakankamai pastatytas“, kad galėtume jį grąžinti?

Žmogus perka namą iš statybininko:

A: su daline apdaila - 85 % namo baigtumas

B: su pilna apdaila - 100 % namo baigtumas

Kas sprendžia: Builder ar Director?

Plane neturi variklio.
Ar `build()` turi suveikti?

- A) Taip

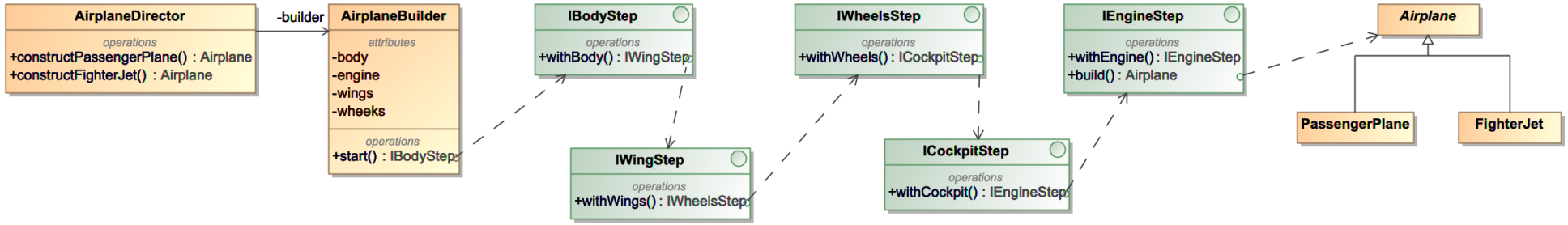
  B) Mesti exception

  C) Engine padaryti mandatory konstruktoriuje

  D) Director apskritai neturi leisti pasiekti `build()`

Kaip užtikrinti, kad lėktuvus gamintų su varinkliais?

Interfeisų grandinės (*Step Builder*)




```
public class AirplaneDirector {
    // Konstruoja standartinį keleivinį lėktuvą
    public Airplane constructPassengerPlane(IEngineStep builder) {
        return builder
            .withEngine("Turbofan Engine")
            .withWings(35.0)
            .withWheels(10)
            .withCockpit("Standard Commercial Cockpit")
            .build(); // Grąžina jau paruoštą Airplane
    }
}
```

Tai kodėl kiekvienai klasei nekuriame Builder?

+ aiškus sudėtingų objektų kūrimas
+ kūrimo logika atskirta nuo produkto

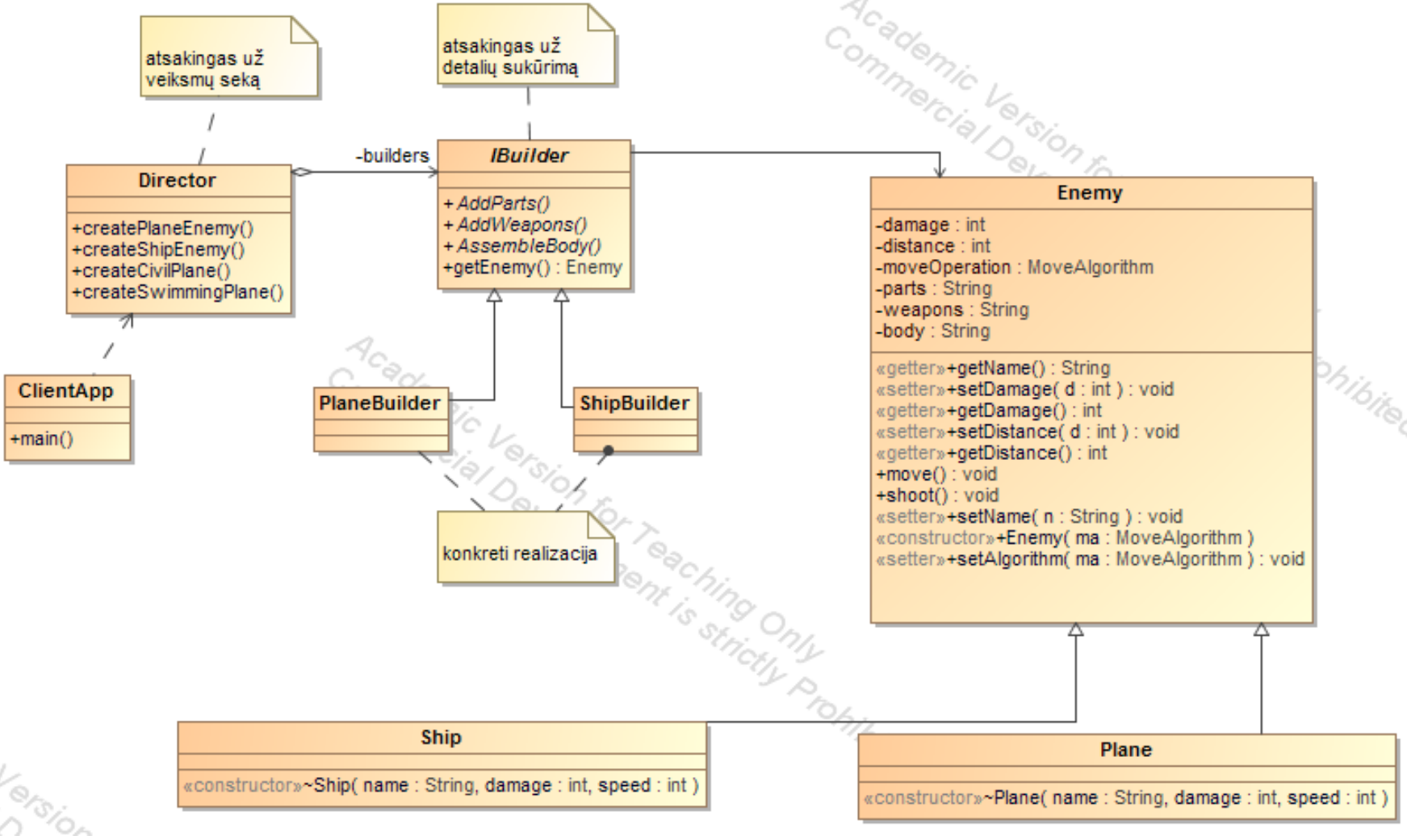
– daugiau klasių
– paprastiems objektams tai overengineering

Ar Point(x, y) reikalingas PointBuilder?

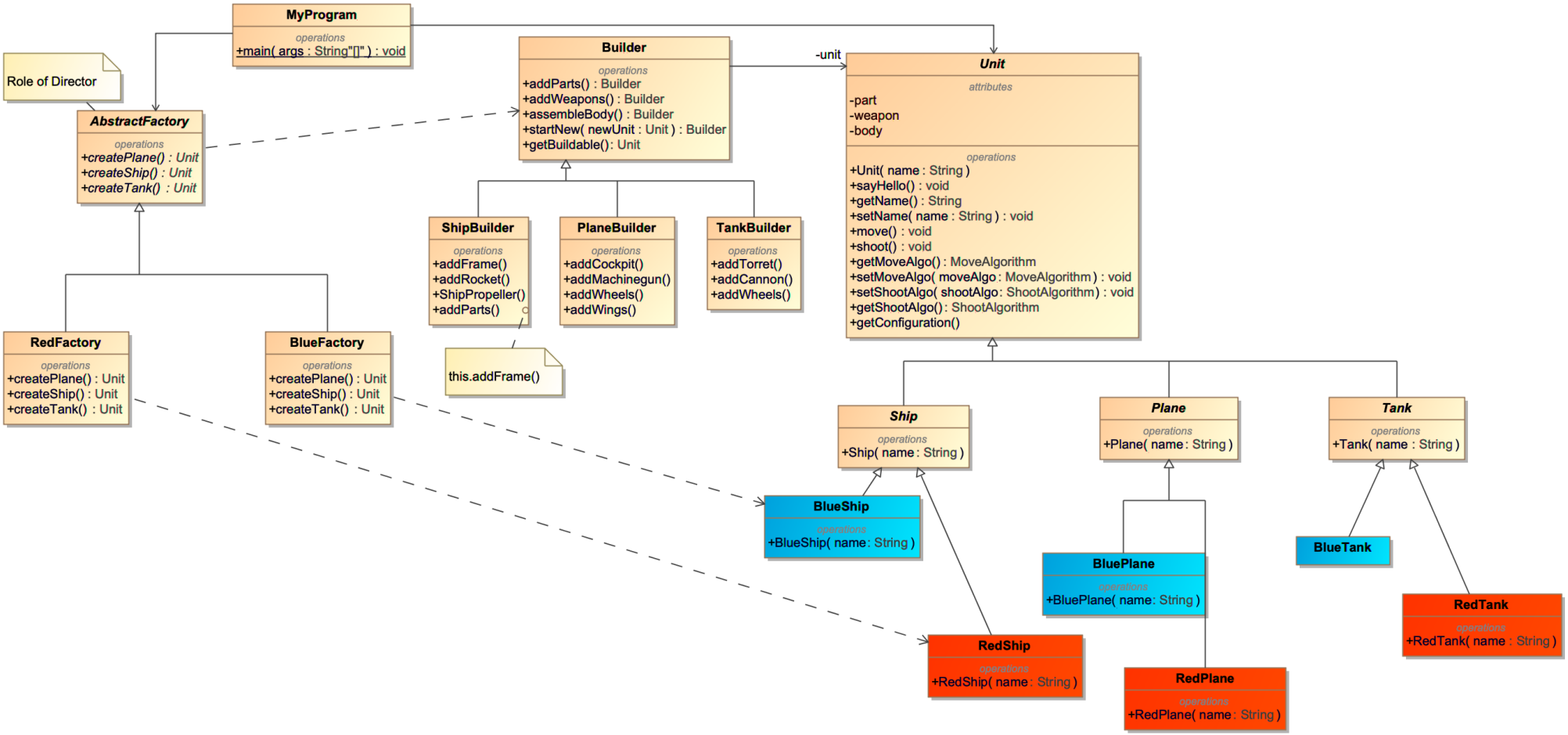
Builder to code – make unit buildable

- Integrate Builder into Abstract Factory
- Create your unit: plane or tank with methods:
- AddParts:
  - addWheels, addWings, ShipPropeller
- AddWeapons:
  - addCannon, addMachinegun, addRocket
- AssembleBody:
  - addTorret, addCockpit, addFrame

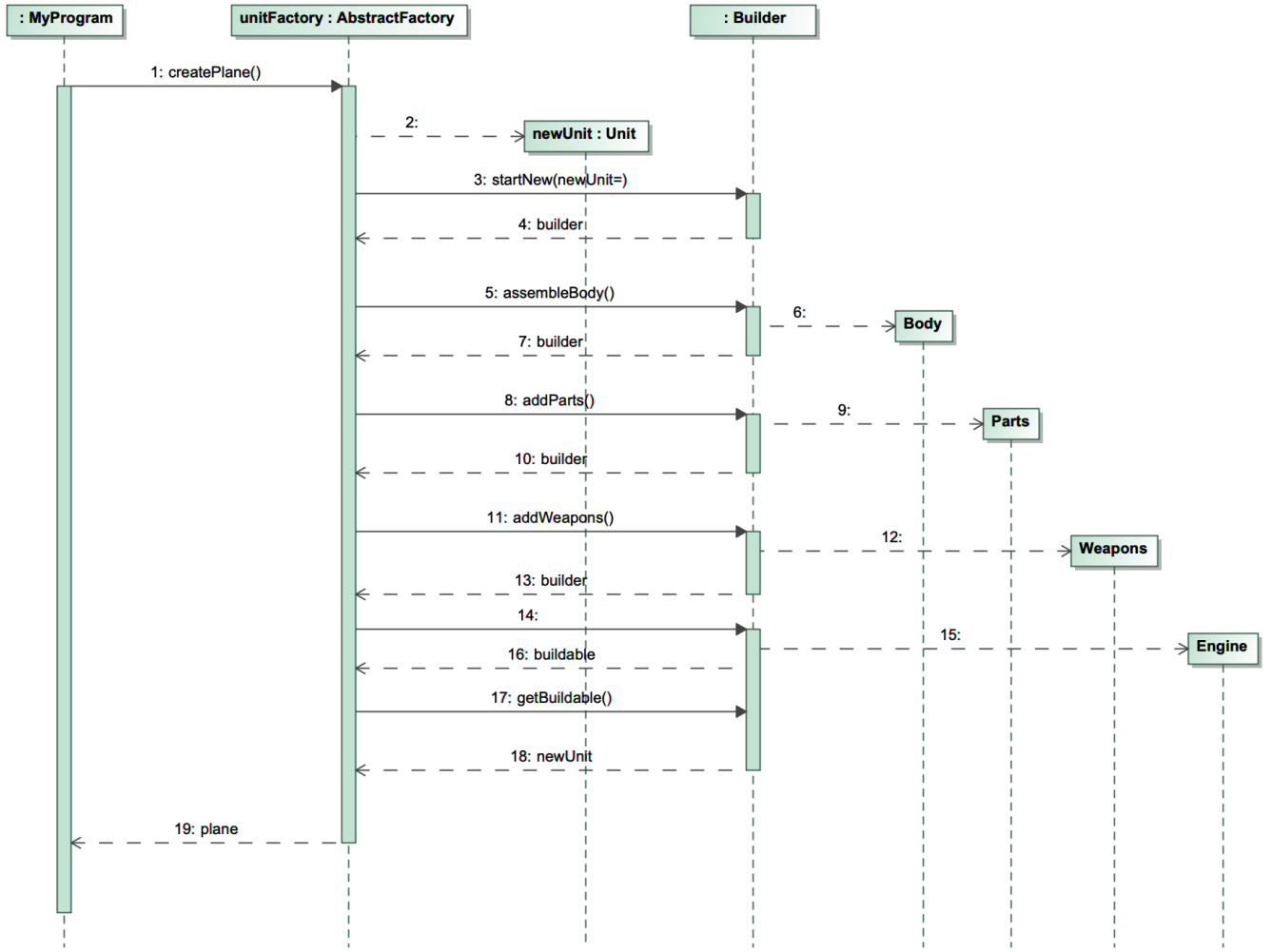
Builder: example



Builder example



Builder sequence



Builder: questions for consideration

- When do you need to create new buildable object?
  - `start()` method can return some configurator (why not buildable object itself?),
  - `reset()` method might help for creation of subsequent object.

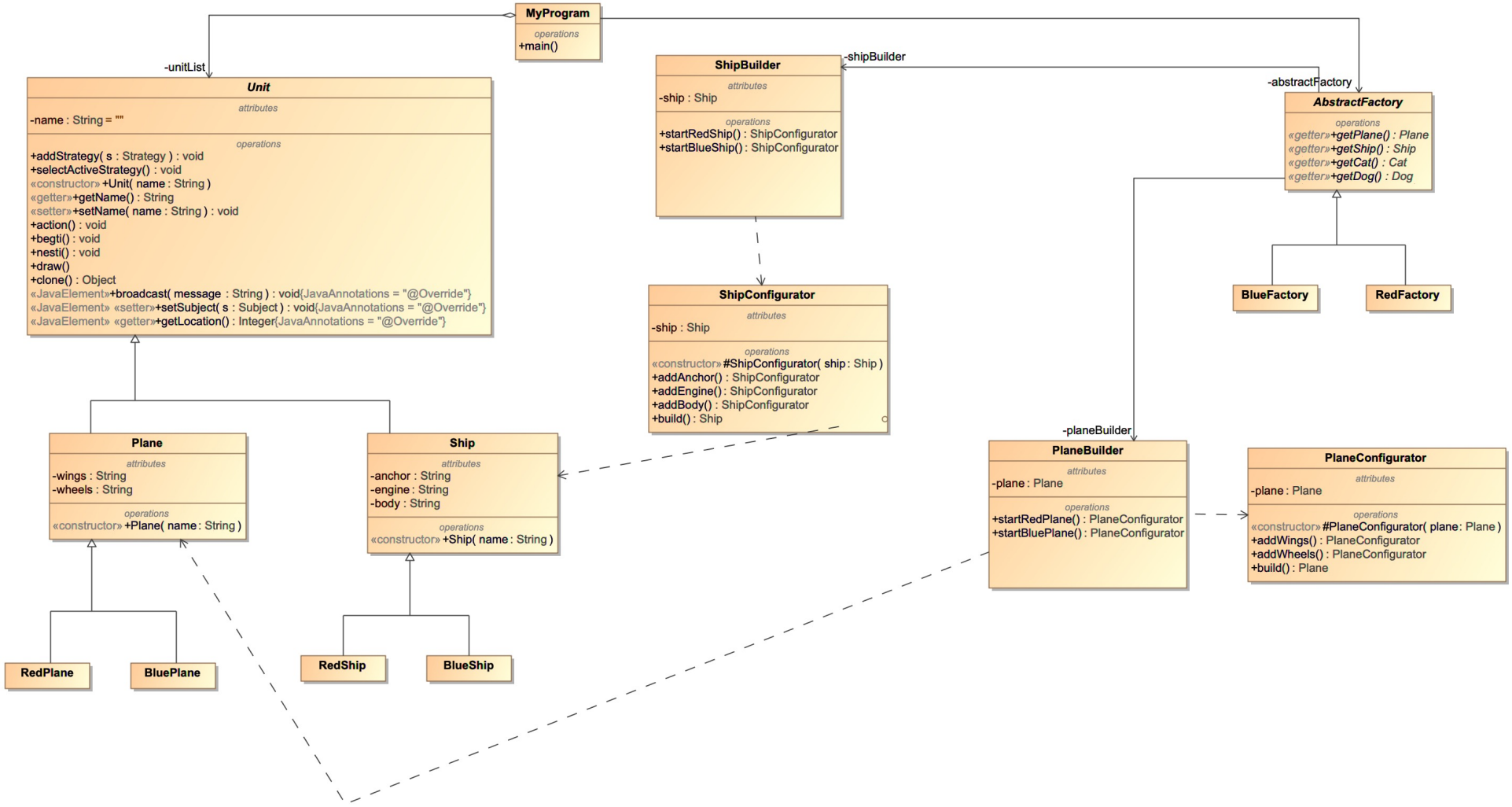
- When do you know if buildable object is ready to be released?
- The Director can ensure the order in which to execute the building steps

- Only `build()` method returns buildable object, while other methods return `<Configurator>` or `IBuilder`

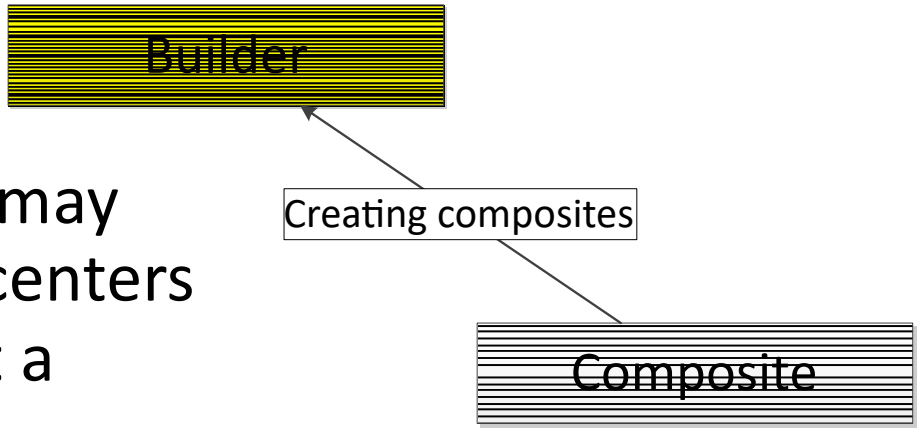
Example :

```
public Plane Configure(Ibuilder builder){
    builder.startBluePlane().addWings().addWheels().build();
}
```

Example of different builders



Builder: related patterns



- Abstract Factory is similar to Builder: may construct complex objects too but it centers on **a single-step** creation process, not a configurable sequence of steps

- The primary difference:
  - Builder pattern is constructing a complex object **step by step** and returns the product at a final step.
  - Abstract Factory's emphasis is on families of product objects and returns the product immediately.
- A Composite is what the builder often builds.

Jau turime puikiai sukonfigūruotą lėktuvą.
Dabar reikia dar 100 tokių.

- Ar visus vėl konstruosime su Builder?
- O ką dar galėtume padaryti?
- Gal kitus tiesiog kopijuokime?

Turime sukurti 500 vienodų Plane. Ką rinktumėtės?

A. new Plane(...) 500 kartų

B. PlaneBuilder.build() 500 kartų

C. Nukopijuoti visus laukus iš jau turimo Plane

D. Nukopijuoti patį Plane

Prototype - Turime gerą egzempliorių → naudojame jį kaip prototipą naujiems objektams.

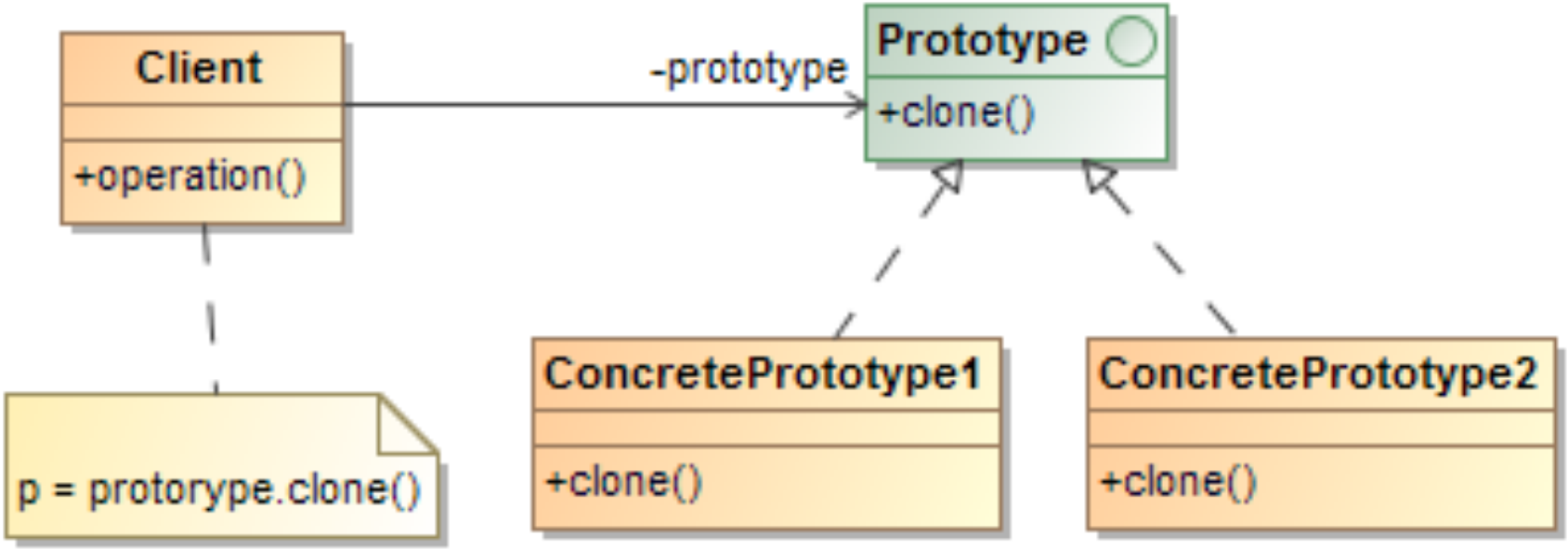
- Type: Creational

- Also Known As:

- Intent: “Specify the kinds of objects to create using a **prototypical instance**, and create new objects by copying this prototype”

Kaip GoF siūlo spręsti 500 lėktuvų problemą?

- **Prototype's structure**



Kaip sukurti kopija?

Copy constructor

```
Plane copy = new
Plane(original);
```

Prototype method

```
Plane copy =
original.clone();
```

Kurie iš šių variantų gali būti naudojami Prototype realizacijai?

A. `Plane p = new Plane(...);` // Constructor
B. `Plane p = planeBuilder.build();` // Builder
C. `Plane p = prototype.clone();` // Prototype method
D. `Plane p = new Plane(prototype);` // Copy constructor

Kas atsitiks, jei klonuojame objektą per abstraktų tipą?

Turime tik Unit. Nežinome, ar tai Plane, Tank ar Ship. Kaip jį nukopijuoti?

```
Unit copy = original.getCopy();   // Plane? Ship? Tank?// Ką rašyti?
Unit prototype = new Plane(...);  // O jeigu unit yra Ship?
//Unit prototype = new Ship(...

if (unit instanceof Plane) ...else
if (unit instanceof Ship
```

Ar Java ir C# jau padeda mums kopijuoti objektus?

Java has Cloneable interface

** clone() method is not defined*

```
1  public class CopyPaste implements Cloneable{
2
3      public CopyPaste getClone() {
4              try {
5                  return (CopyPaste) super.clone();
6              } catch (CloneNotSupportedException e) {
7                  e.printStackTrace();
8                  return this;
9              }
10
11         }
12 }
```

In C# you can use MemberwiseClone() method

```
public override Prototype Clone()
{
  return (Prototype)this.MemberwiseClone();
}
```

Taigi, kada mums praverčia Prototype?

- objektą **brangu ar sudėtinga sukurti**;
- jau turime **tinkamai sukonfigūruotą egzempliorių**;
- reikia daug **panašių objektų**;
- norime pakeisti tik **kelias kopijos savybes**.

- Mūsų žaidime: vieną Orc sukonfigūravome → 500 kartų klonavome → pakeitėme poziciją.

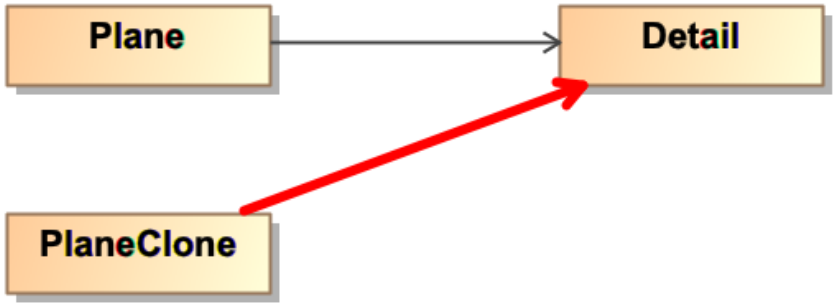
Jei po clone pakeitėme pusę objekto, ar Prototype dar buvo geras pasirinkimas?

- Kada kopija nustoja būti kopija?

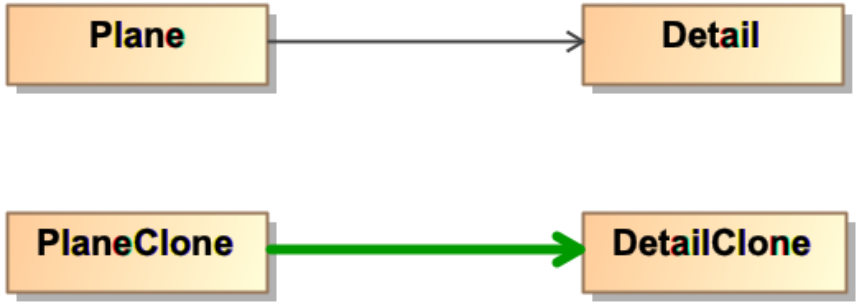
- Kuo labiau naujas objektas skiriasi nuo prototipo, tuo mažiau naudos gauname iš kopijavimo ir tuo labiau gal verta Builder / Factory / normal construction.

Nukopijavome Plane. O ar nukopijavome jo Weapon?

Shallow Copy



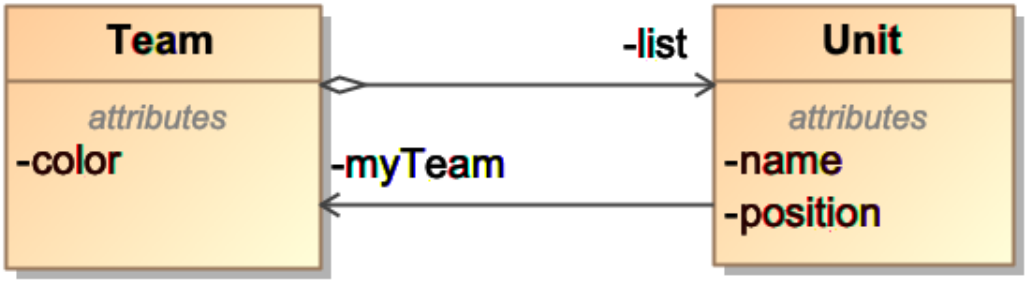
Deep Copy



Hybrid / Selective Copy

- **Kodėl verta taikyti mišrų požiūrį?**
- **Optimizacija ir atminties taupymas:** Jei aklai darytume *Deep Copy* viskam iš eilės, be reikalo kopijuotume nekintamus objektus, konfigūracijas ar sujungimo tarnybas (*services*), taip apkraudami RAM atmintį ir procesorių.
- **Saugumas:** Visiems mutuojamiems duomenims (kur vieno lėktuvo pakeitimai negali paveikti kito) atliekamas *Deep Copy*, užkertant kelią kritinėms klaidoms.
- **Taisyklė trumpai:**
- Kopijuojant objektą, **darykite Deep Copy tik tiems laukams, kurių būsena bus keičiama atskirai**, o viskam kitam (statiniams duomenims, servisams, immutable objektams) drąsiai taikykite **Shallow Copy**.

Reikia komandos ir jos unit’ų deep copy, kas nutiks?



A) Nukopijuos Team ir visus Unit be problemų

B) Nukopijavus Unit, bus kopijuojamas ir Team dar kartą

C) Java automatiškai atpažins ciklą

D) Priklauso nuo Deep Copy realizacijos

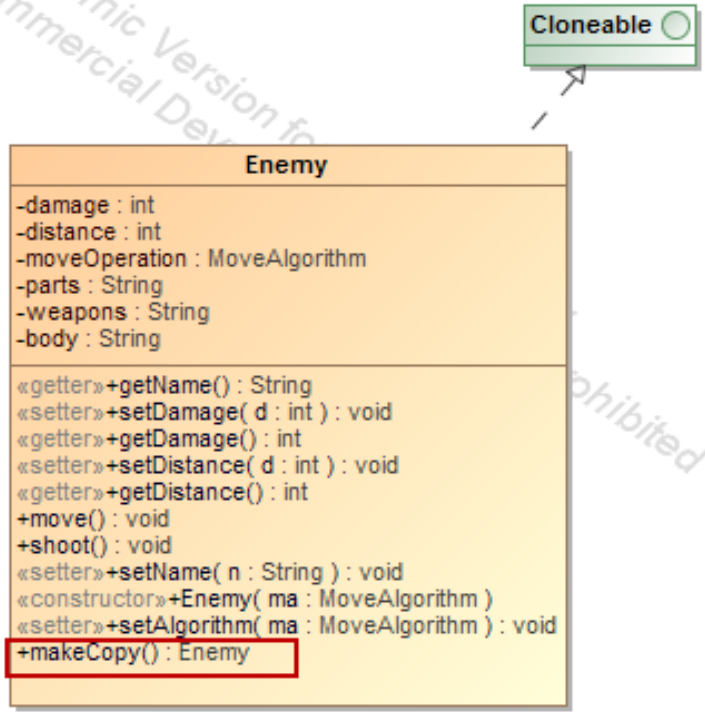
Kodėl visur nenaudojame Prototype?

+ nereikia kartoti sudėtingos inicializacijos
+ labai patogu kurti daug panašių objektų

– sudėtinga nuspręsti, ką kopijuoti
– shallow/deep copy gali sukelti netikėtų problemų
– sudėtingi objektų grafai / circular references komplikuoja kopijavimą

Ar verta klonuoti Point(x, y)?

Prototype : example




```java
/**
 * @(#) Enemy.java
 */
public class Enemy implements Soldier, Cloneable
{
```

```java
    //Prototype design pattern
    public Enemy makeCopy(){
        try{
            return (Enemy) this.clone();
        }catch(CloneNotSupportedException ex){
            ex.printStackTrace();
            return this;
        }
    }
```

```java
        //Prototype example
        Enemy clonedSubmarine = submarine.makeCopy();
        System.out.println("Submarine hashcode : " + System.identityHashCode(submarine));
        System.out.println("Cloned sub hashcode: " + System.identityHashCode(clonedSubmarine));

        //Prototype influence on Observer example
        clonedSubmarine.setCommander(cmd);
        cmd.notifySoldiers("Order: Fire!");
```

Reikia 100 laivų: Builder ar Prototype?

Builder

↓

pirmas Ship

↓

Prototype

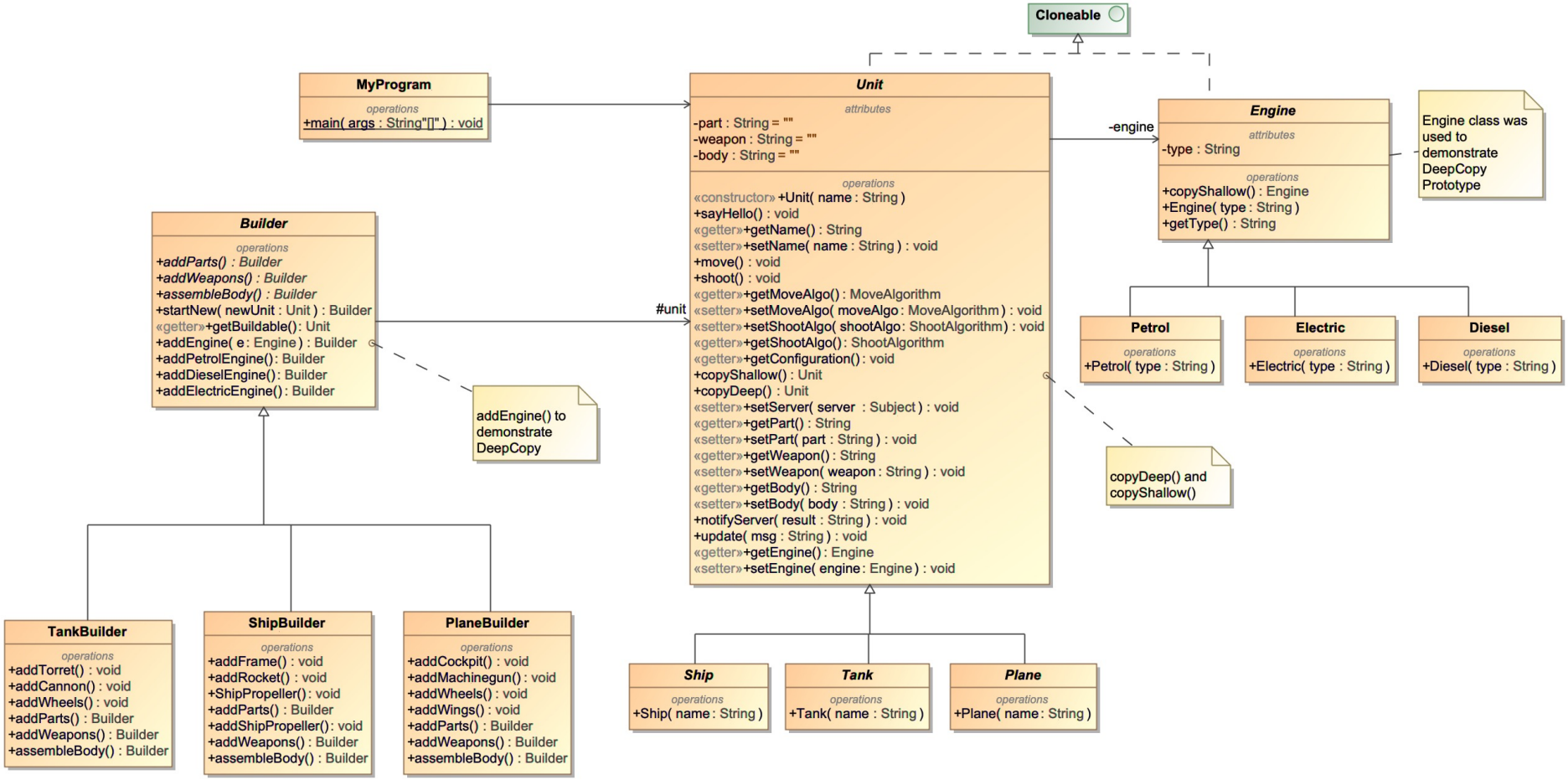
↙ ↓ ↓ ↘

Ship Ship Ship...

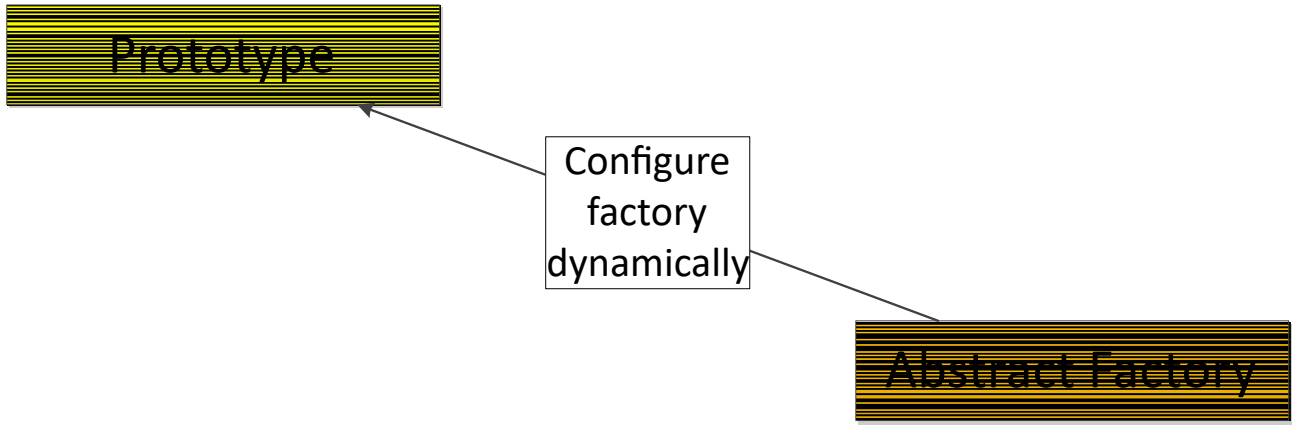
Prototype : example to code

- Constructor / Builder / clone / copy constructor?
- Make several clones of ship and/or plane
- Compare creating efficiency:
  - By clone()
  - By copy constructor
  - By streaming

Builder and Prototype example



Prototype: related patterns



Ką šiandien išmokome?

Builder

Kuriame sudėtingą objektą žingsnis po žingsnio.

`Plane → body → engine → wings → weapon → build()`

Prototype

Naują objektą sukuriame kopijuodami jau turimą.

`Plane → clone() → Plane'`

Klausimai?

Singleton ir Observer realizacijos pavyzdžiai

- Norėtųsi atkreipti ir į juos dėmesį